

11. Automating Application Deployment Using a CI/CD Pipeline

Step 1: Create an IAM Role for EC2 and AWS CodeDeploy

- Go to AWS IAM Console.
- Navigate to Roles and click Create Role.
- Select AWS Service as the trusted entity type.
- Choose EC2 as the use case.
- Click Next and add the permission AmazonS3ReadOnlyAccess.
- Provide a role name and click Create Role.

Step 2: Create an IAM Role for AWS CodeDeploy

1. Repeat Step 1 but select CodeDeploy as the use case.
2. Attach AWSCodeDeployRole permissions.
3. Name the role and click Create Role.

Step 3: Launch an EC2 Instance

1. Go to EC2 Dashboard and launch an instance.
2. Select Amazon Linux as the AML.
3. Enable SSH, HTTP, and HTTPS.
4. Click Launch Instance.

Step 4: Connect to EC2 and Install Dependencies

Connect to EC2 via SSH:

css

Copy

Edit

```
ssh -i your-key.pem ec2-user@your-public-ip
```

1. Switch to root

user: nginx

Copy

Edit

sudo

su

2. Update system

packages: sql

CopyEdit

```
sudo yum update -y
```

3. Install Ruby and
Wget: nginx
CopyEdit
sudo yum install ruby

-y sudo yum install

wget -y

Step 5: Install AWS CodeDeploy Agent

Download the installation script:

bash
Copy
Edit
wget https://aws-codedeploy-us-east-1.s3.amazonaws.com/latest/install

1. Make the script executable: bash
CopyEdit
chmod +x ./install
2. Install CodeDeploy Agent: arduino
CopyEdit
sudo ./install auto
3. Check the
status: lua
CopyEdit
sudo service codedeploy-agent status

Step 6: Attach IAM Role to EC2

1. Go to EC2 Dashboard > Select the Instance.
2. Click Actions > Security > Modify IAM Role.
3. Attach the previously created IAM Role.

Step 7: Configure Security Group

1. Go to EC2 Dashboard.
2. Click on Security Groups.
3. Edit Inbound Rules:
 - Type: Custom TCP
 - Port: 4000 (or any required port)
 - Source: Anywhere (0.0.0.0/0 for IPv4)
4. Click Save Rules.

Step 8: Create a CodePipeline

1. Go to AWS CodePipeline and click Create Pipeline.

2. Enter a Pipeline Name.
3. Choose GitHub as the source.
4. Click Connect to GitHub and authenticate.

Step 9: Configure CodeBuild (Optional)

1. If required, create a Build Project.
2. Add a buildspec.yaml file in your project root.

Step 10: Create a Deployment Group

1. Go to AWS CodeDeploy.
2. Click Create Application.
3. Create a Deployment Group.
4. Select EC2 Instances and provide key-value tags.
5. Uncheck the Load Balancer option.

Step 11: Add Deploy Stage

1. Return to CodePipeline.
2. Select the previously created Application Name and Deployment Group.

Step 12: Review and Create Pipeline

1. Review all settings.
2. Click Create Pipeline.
3. The CI/CD pipeline is now set up and running.

P10 : AWS CI/CD Pipeline

Step 1: Create IAM Role for EC2 and AWS CodeDeploy

- Navigate to IAM service.
- Then go to roles and create a new role.
- Select trusted entity type as AWS Service and use case as EC2

Select trusted entity [Info](#)

Trusted entity type

☒ **AWS service**
 Allow AWS services like EC2, Lambda, or others to perform actions in this account.

☐ **AWS account**
 Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.

☐ **Web identity**
 Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.

☐ **SAML 2.0 federation**
 Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.

☐ **Custom trust policy**
 Create a custom trust policy to enable others to perform actions in this account.

Use case

Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case

EC2

Choose a use case for the specified service.

Use case

☒ **EC2**
 Allows EC2 instances to call AWS services on your behalf.

☐ **EC2 Role for AWS Systems Manager**

Step 2: Add permissions To IAM Role

- Select AmazonS3ReadOnlyAccess permission. It will allow our EC2 instance to access stored artifacts from the Amazon S3 bucket.

Add permissions [Info](#)

Permissions policies (1/884) [Info](#)

Choose one or more policies to attach to your new role.

Filter by Type
 All types

☒	Policy name ↗	Type
☒	 AmazonS3ReadOnlyAccess	AWS managed

► **Set permissions boundary - optional**

Step 3: Creating The Role For AWS CodeDeploy

- Provide the Name, review and Click on Create for creating the Role.
- Select an appropriate role name and click on create role.

Name, review, and create

Role details

Role name

Enter a meaningful name to identify this role.

EC2RoleAWSCodeDeploy-1

Maximum 64 characters. Use alphanumeric and '+=, @-_' characters.

Description

Add a short explanation for this role.

Allows EC2 instances to call AWS services on your behalf.

Maximum 1000 characters. Use alphanumeric and '+=, @-_' characters.

Step 4: Creating New Service Role For CodeDeploy

- Create a new service role for CodeDeploy and attach AWSCodeDeployRole policy which will provide the permissions for our service role to read tags of our EC2 instance, publish information to Amazon SNS topics and much more task.
- Repeat the Above 3 steps again with trusted entity type AWS Service, use case CodeDeploy.

Select trusted entity [Info](#)

Trusted entity type

☒ AWS service

Allow AWS services like EC2, Lambda, or others to perform actions in this account.

☐ AWS account

Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.

☐ Web identity

Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.

☐ SAML 2.0 federation

Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.

☐ Custom trust policy

Create a custom trust policy to enable others to perform actions in this account.

Use case

Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case

CodeDeploy

Choose a use case for the specified service.

Use case

☒ CodeDeploy

Allows CodeDeploy to call AWS services such as Auto Scaling on your behalf.

☐ CodeDeploy for Lambda

Allows CodeDeploy to route traffic to a new version of an AWS Lambda function version on your behalf.

☐ CodeDeploy - ECS

Allows CodeDeploy to read S3 objects, invoke Lambda functions, publish to SNS topics, and update ECS services on your behalf.

- Add AWSCodeDeployRole permissions to this creating Role

Add permissions Info

Permissions policies (1) Info

The type of role that you selected requires the following policy.

Policy name	Type
 AWSCodeDeployRole	AWS managed

► **Set permissions boundary - optional**

- Provide the Name, review and create the role.

Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.

Maximum 64 characters. Use alphanumeric and '+=, @, _' characters.

Description
Add a short explanation for this role.

Maximum 1000 characters. Use alphanumeric and '+=, @, _' characters.

Step 5: Launch An Linux EC2 instance

- Select the instance with AMI such as "Amazon Linux" and connect to CLI Console.
- Switch to root user from ec2-user to gain admin access power by using following command "sudo su" in Linux.

```
sudo su
```

Step 6: Update The Packages

- The command "sudo yum update" is used in Amazon Linux, CentOS, and Red Hat Linux distributions to update installed packages on your system to their latest available versions.

```
sudo yum update
```

Step 7: Install The Ruby And Wget Software

- The command 'sudo yum install ruby' is used to install the Ruby programming software using the YUM package manager.

```
sudo yum install ruby
```

- The command sudo yum install wget is used to install the "wget" package on a system running Amazon Linux, CentOS, or other Red Hat-based Linux distributions that use the YUM package manager.

```
sudo yum install wget
```

Step 8: Download CodeDeploy Agent Script

- Downloading the AWS CodeDeploy agent installation script from the AWS S3 bucket is an essential step in setting up AWS CodeDeploy for your infrastructure.
- The CodeDeploy agent is a lightweight, scalable software component that enables AWS CodeDeploy to deploy and manage applications on your EC2 instances or on-premises servers.

```
wget https://aws-codedeploy-us-east-1.s3.amazonaws.com/latest/install
```

Step 9: Run Installation Script

- The command `chmod +x ./install` is used to make a file executable in a Unix-like operating system, including Linux.

```
chmod+x./install
```

The command `'sudo ./install auto'` is likely used to run an installation script with superuser (administrator) privileges and pass the "auto" argument to the script.

```
sudo./install auto
```

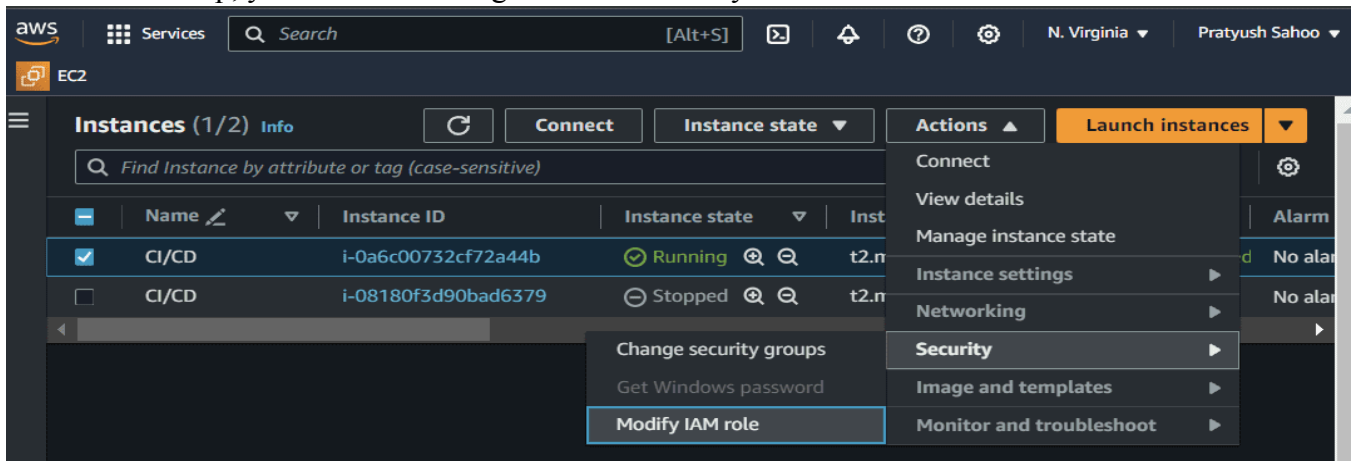
Step 10: Check CodeDeploy Agent Status

- The command `sudo service codedeploy-agent status` is used to check the status of the AWS CodeDeploy agent running on your system.

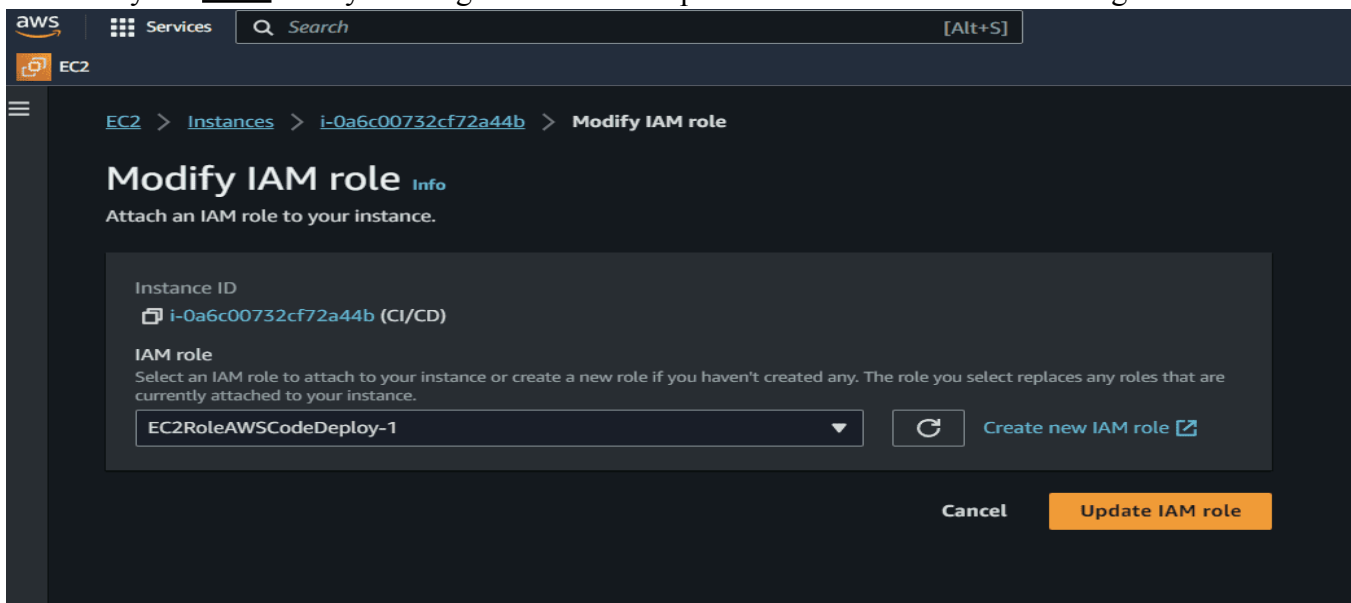
```
Sudo service codedeploy-agent status
```

Step 11: Modifying IAM Role

- After running the following commands, select the instance and click on "Actions", then click on "Security" and click on "Modify IAM Role". Then choose the above created IAM Role and click on "Update IAM Role".
- After this step, your EC2 instance gets attached with your above created IAM Role.



- Modify the IAM role by clicking on the button Update IAM role as shown in the figure.



Step 12: Finalizing The Configuration

After this process, go to the console where your instance is connected and run the command "exit" to

exit from the root folder and go back to the EC2 folder. Make a directory on the EC2 folder named "server", this is the directory where my source code will be deployed.

The screenshot shows the AWS Management Console interface with a terminal window open. The terminal displays the following commands and output:

```
Complete!
I, [2023-10-22T18:30:38.357273 #30431] INFO -- : Update check complete.
I, [2023-10-22T18:30:38.357450 #30431] INFO -- : Stopping updater.
[root@ip-172-31-27-184 ec2-user]# sudo service codedeploy-agent status
/opt/codedeploy-agent/vendor/gems/gli-2.11.0/lib/gli/commands/help_modules/global_help_format.rb:37: warning: Passing safe_level with the 2nd argument of ERB.new is deprecated. Do not use it, and specify other arguments as keyword arguments.
/opt/codedeploy-agent/vendor/gems/gli-2.11.0/lib/gli/commands/help_modules/global_help_format.rb:37: warning: Passing trim_mode with the 3rd argument of ERB.new is deprecated. Use keyword argument like ERB.new(str, trim_mode: ...) instead.
/opt/codedeploy-agent/vendor/gems/gli-2.11.0/lib/gli/commands/help_modules/command_help_format.rb:27: warning: Passing safe_level with the 2nd argument of ERB.new is deprecated. Do not use it, and specify other arguments as keyword arguments.
/opt/codedeploy-agent/vendor/gems/gli-2.11.0/lib/gli/commands/help_modules/command_help_format.rb:27: warning: Passing trim_mode with the 3rd argument of ERB.new is deprecated. Use keyword argument like ERB.new(str, trim_mode: ...) instead.
The AWS CodeDeploy agent is running as PID 30588
[root@ip-172-31-27-184 ec2-user]# ls
install
[root@ip-172-31-27-184 ec2-user]# exit
exit
[ec2-user@ip-172-31-27-184 ~]$ ls
install
[ec2-user@ip-172-31-27-184 ~]$ mkdir server
[ec2-user@ip-172-31-27-184 ~]$ ls
install server
[ec2-user@ip-172-31-27-184 ~]$
```

Below the terminal window, the instance details are shown:

i-0a6c00732cf72a44b (CI/CD)

PublicIPs: 54.163.18.150 PrivateIPs: 172.31.27.184

- Then after doing the above process, come back to the running instances list.
- Select your currently created running instance and go to the "Security" section present at the end of the page.
- Click on the link present under the "Security Groups". After redirecting to the required page, click on "Edit Inbound rules" under the section of "Inbound rules" present at the end of the page.
- Then add a rule, select a port range of your choice and select the source as "Anywhere-IPv4" from the dropdown menu and then click on "Save rules".
- Basically, let me give you an overview of what we are actually doing here. In brief, when you add an inbound rule to a security group for an instance with a port range (in my case, it was 4000) and set the source to "Anywhere-IPv4," you are allowing any computer or device on the internet to connect to your instance through port 4000.
- This is like opening a door (port 4000) on your server and letting anyone from anywhere access the service or application running on that port.